Universidad de Granada

ETSIIT Informática y Telecomunicación

Dpto. de Ciencias de la Computación e Inteligencia Artificial

Metaheurísticas

Práctica 2

Técnicas de Búsqueda basadas en Poblaciones
para el Problema de Portfolio

Curso 2025–2026

Nombre: Jose Angel Carretero Montes
DNI: 25628892S
Email: joseangel1@correo.ugr.es
Grupo: Grupo 3
Horario: Martes 15:30–17:30

Índice

1. Descripción del Problema **2**

2. Aplicación de los Algoritmos al Problema **2**
2.1. Representación de soluciones 2
2.2. Generación aleatoria de soluciones 3
2.3. Operador de reparación (`fix`) 3
2.4. Operador de selección: torneo binario 3
2.5. Operadores de cruce 3
2.6. Operador de mutación 4

3. Algoritmos Genéticos y Meméticos **4**
3.1. Algoritmo Genético Generacional (AGG) 4
3.2. Algoritmo Genético Estacionario (AGE) 5
3.3. Algoritmos Meméticos (AM): Integración de Búsqueda Local 6

4. Estructura del Código y Manual de Usuario **9**
4.1. Estructura de directorios 9
4.2. Compilación y ejecución 9

5. Experimentos y Resultados **9**
5.1. Casos del problema 9
5.2. Resultados 10
5.3. Boxplots 10

6. Análisis de Resultados **13**
6.1. Impacto del operador de cruce: AGG-Arit vs AGG-BLX 13
6.2. Impacto del operador de cruce: AGE-Arit vs AGE-BLX 14
6.3. Comparación de esquemas: AGG vs AGE 14
6.4. Algoritmos Meméticos: AM-All vs AM-Rand vs AM-Best 14
6.5. Búsqueda Local vs Algoritmos Genéticos 15
6.6. Capacidad de generalización 15
6.7. Variabilidad y coste computacional 16
6.8. Conclusiones generales 16

1. Descripción del Problema

El **Problema de Portfolio** consiste en determinar la asignación óptima de capital entre N activos financieros con el objetivo de maximizar el retorno esperado minimizando simultáneamente el riesgo.

Representación de los datos

Para cada activo $i \in \{1, \ldots, N\}$ y día $t \in \{1, \ldots, T\}$ se dispone del retorno diario μ_{ti}, calculado como el ratio de mejora relativa del valor:

$$\mu_{ti} = \frac{V_{ti} - V_{(t+1)i}}{V_{ti}}$$

Los datos empleados cubren el periodo 2015–2024 para entrenamiento y el año 2025 para evaluación de generalización.

Función objetivo

Dado un vector de pesos $w = (w_1, \ldots, w_N)$, la calidad de la cartera se mide como:

$$\text{fitness}(w) = \text{Beneficio}(w) - \lambda \cdot \text{Riesgo}(w)$$

donde:

$$\text{Beneficio}(w) = \sum_{i=1}^{N} w_i \cdot \sum_{t=1}^{T} \log(1 + \mu_{ti}) \qquad \text{Riesgo}(w) = \sqrt{\sum_{i=1}^{N} \sum_{j=1}^{N} w_i \cdot \Sigma_{ij} \cdot w_j}$$

siendo Σ_{ij} la covarianza entre activos i y j, y $\lambda = 500$ el factor de penalización del riesgo.

Restricciones

$$\sum_{i=1}^{N} w_i = 1 \qquad w_i = 0 \ \vee \ lo \leq w_i \leq hi \quad \forall i \in 1 \ldots N$$

Los rangos $[lo, hi]$ varían según el conjunto de datos (véase Sección 5).

2. Aplicación de los Algoritmos al Problema

2.1. Representación de soluciones

Una solución queda representada como un vector $w \in \mathbb{R}^N$ donde cada componente w_i indica la fracción del capital invertida en el activo i. La restricción de cardinalidad implica que w_i puede ser exactamente cero (no se invierte en esa empresa) o estar en el intervalo $[lo, hi]$.

2.2. Generación aleatoria de soluciones

La generación de la solución inicial es la misma que en la Práctica 1: se elige un subconjunto aleatorio de empresas activas, se les asigna un peso aleatorio en $[lo, hi]$, se normaliza y se aplica el operador de reparación.

2.3. Operador de reparación (`fix`)

El operador de reparación es el mismo que el empleado en la Práctica 1 (tres fases: normalización, recorte de rango, redistribución iterativa). Es especialmente importante en los algoritmos genéticos ya que los operadores de cruce pueden generar soluciones que violen las restricciones.

2.4. Operador de selección: torneo binario

En ambos esquemas (generacional y estacionario) se usa el **torneo binario**: se eligen dos individuos al azar de la población y se devuelve el de mayor fitness.

Algorithm 1: Torneo binario

Entrada: población P
Salida: individuo ganador
$i_1 \leftarrow$ índice aleatorio en $[0, |P| - 1]$;
$i_2 \leftarrow$ índice aleatorio en $[0, |P| - 1]$;
si $P[i_1].\mathit{fitness} \geq P[i_2].\mathit{fitness}$ **entonces**
  devolver $P[i_1]$
devolver $P[i_2]$

2.5. Operadores de cruce

Se emplean dos operadores para representación real.

Cruce Aritmético

Para cada posición i se genera un $r_i \in [0, 1]$ independiente:

$$\text{Off}_i^1 = r_i \cdot S_i^1 + (1 - r_i) \cdot S_i^2 \qquad \text{Off}_i^2 = r_i \cdot S_i^2 + (1 - r_i) \cdot S_i^1$$

Cruce BLX-α

Para cada posición i, siendo $I_i = |S_i^1 - S_i^2|$:

$$hmin_i = \text{mín}(S_i^1, S_i^2) - \alpha \cdot I_i \qquad hmax_i = \text{máx}(S_i^1, S_i^2) + \alpha \cdot I_i$$

Ambas soluciones hijas se generan de igual forma: cada gen del hijo se muestrea uniformemente en $[\text{máx}(0, hmin_i), \text{mín}(hi, hmax_i)]$, con $\alpha = 0{,}3$.

Tras ambos cruces se aplica **fix** a los hijos generados.

2.6. Operador de mutación

Para Algoritmos Genéticos (AG)

Con probabilidad $p_m = 0{,}1$ **por individuo**, se selecciona un gen aleatorio y se reemplaza por un valor uniforme en $[lo, hi]$, seguido de **fix**. Solo cuenta como evaluación si efectivamente se muta.

Para Algoritmos Meméticos (AM)

Con probabilidad $p_m = 0{,}1/N$ **por gen**, cada gen se reemplaza independientemente por un valor uniforme en $[lo, hi]$, seguido de **fix**. Solo cuenta como evaluación si efectivamente se muta algún gen.

La mutación por gen en AMs favorece una exploración más uniforme del espacio, mientras que la mutación por individuo en AGs es más selectiva.

3. Algoritmos Genéticos y Meméticos

3.1. Algoritmo Genético Generacional (AGG)

El AGG mantiene una población completa de $N_p = 100$ individuos que se reemplaza generación a generación. Incorpora elitismo para no perder la mejor solución encontrada.

Parámetros

- Tamaño de población: $N_p = 100$
- Probabilidad de cruce: $p_c = 0{,}8$
- Probabilidad de mutación: $p_m = 0{,}1$ (por individuo)
- Criterio de parada: 10 000 evaluaciones de la función objetivo

Esquema de evolución y reemplazamiento

Algorithm 2: AGG (esquema generacional con elitismo)

Entrada: problema, maxEvals
Salida: mejor solución encontrada

Inicializar población P con N_p individuos aleatorios;
$evals \leftarrow N_p$;
mientras $evals < maxEvals$ **hacer**
  $P' \leftarrow \emptyset$;
  para $i \leftarrow 1$ **to** $N_p/2$ ***y*** $evals < maxEvals$ **hacer**
    $p_1 \leftarrow$ `torneo(`P`)`; $p_2 \leftarrow$ `torneo(`P`)`;
    si $rand() \leq 0{,}8$ **entonces**
      $(h_1, h_2) \leftarrow$ `cruzar(`p_1, p_2`)`; $evals \leftarrow evals + 2$;
    si no si $h_1 \leftarrow p_1$; $h_2 \leftarrow p_2$;
     entonces
      `mutar`
    h_1; `mutar(`h_2`)`;
    $P' \leftarrow P' \cup \{h_1, h_2\}$;
  `// Elitismo: conservar el mejor padre si la nueva generación es peor`
  si máx(P').*fitness* $<$ máx(P).*fitness* **entonces**
    mín$(P') \leftarrow$ máx(P);
  $P \leftarrow P'$;
devolver máx(P)

Se implementan dos variantes: **AGG-Arit** (cruce aritmético) y **AGG-BLX** (cruce BLX-α), siendo esa la única diferencia entre ellas.

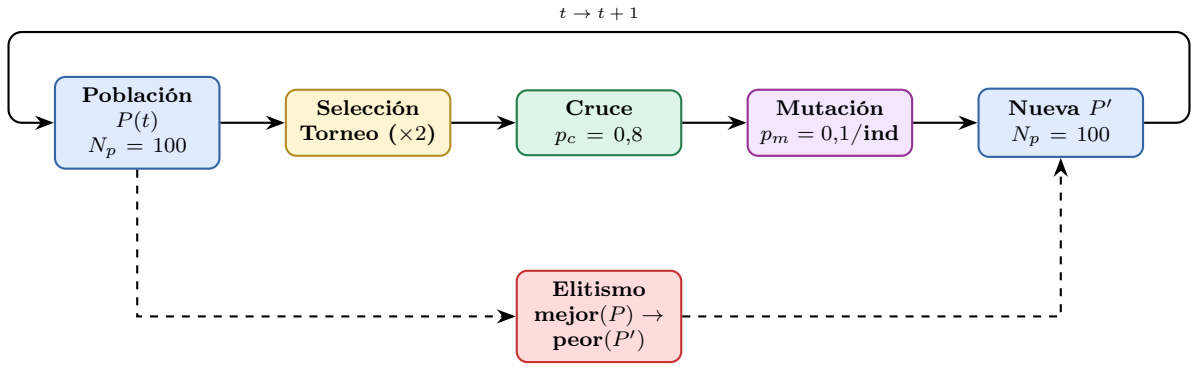


Figura 1: Ciclo evolutivo del AGG. Las líneas discontinuas muestran el flujo de elitismo.

3.2. Algoritmo Genético Estacionario (AGE)

El AGE genera solo dos descendientes por iteración y los inserta en la población reemplazando a los peores individuos si mejoran. La población apenas varía en cada paso, lo que lo hace más conservador.

Parámetros

- Tamaño de población: $N_p = 100$
- Probabilidad de cruce: $p_c = 1{,}0$ (siempre se cruzan)
- Probabilidad de mutación: $p_m = 0{,}1$ (por individuo)
- Criterio de parada: 10 000 evaluaciones de la función objetivo

Esquema de evolución y reemplazamiento

Algorithm 3: AGE (esquema estacionario)

Entrada: problema, maxEvals
Salida: mejor solución encontrada

```
Inicializar población P con N_p individuos aleatorios;
evals ← N_p;
mientras evals < maxEvals hacer
    p_1 ← torneo(P);   p_2 ← torneo(P);
    (h_1, h_2) ← cruzar(p_1, p_2);   evals ← evals + 2;
    mutar(h_1);   mutar(h_2);
    // Reemplazamiento competitivo: cada hijo desafía al peor
    si h_1.fitness > mín(P).fitness entonces
        mín(P) ← h_1;
    si h_2.fitness > mín(P).fitness entonces
        mín(P) ← h_2;
devolver máx(P)
```

Se implementan dos variantes: **AGE-Arit** y **AGE-BLX**.

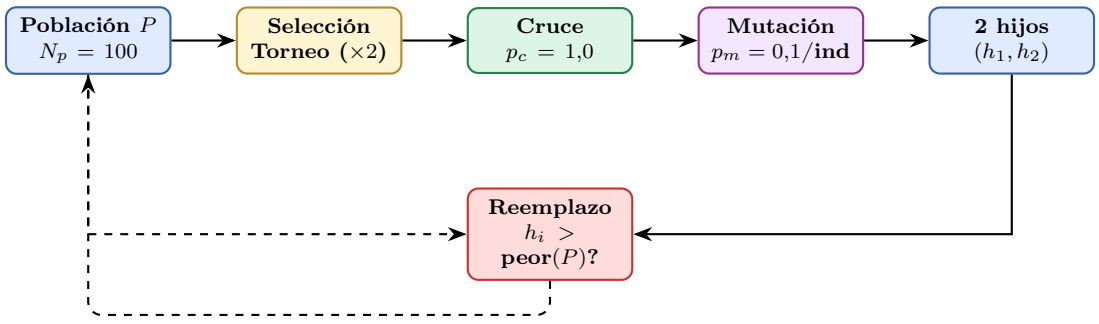


Figura 2: Ciclo evolutivo del AGE. Las líneas discontinuas muestran la consulta del peor individuo de P y la actualización en sitio: solo 0–2 miembros cambian por iteración.

3.3. Algoritmos Meméticos (AM): Integración de Búsqueda Local

Los Algoritmos Meméticos combinan la exploración global de los AGs con la explotación local de la Búsqueda Local (BL). La BL se aplica **cada 10 generaciones** a subconjuntos de la población según tres estrategias de selección.

Los AMs se basan en el AGG que mejores resultados ha proporcionado, que según los experimentos es **AGG-BLX** (véase Sección 5).

Búsqueda Local integrada

La BL empleada es la **primero el mejor** (first improvement) con el operador de movimiento del portfolio: transferir una fracción $p = 0{,}15$ del peso de un activo origen a un activo destino. Parámetro: máximo 100 evaluaciones por llamada. Esta es la misma BL de la Práctica 1 pero con fracción de movimiento reducida del 40 % al 15 %. El contador total de evaluaciones incluye las realizadas durante la BL.

Parámetros comunes de AMs

- Tamaño de población: $N_p = 100$
- Esquema base: AGG-BLX (cruce BLX-α, $p_c = 0{,}8$)
- Probabilidad de mutación: $p_m = 0{,}1/N$ (por gen)
- Aplicación de BL: cada 10 generaciones
- Presupuesto por llamada BL: mín(100, maxEvals $-$ *evals*) evaluaciones
- Criterio de parada: 10 000 evaluaciones (incluyendo las de la BL)

AM-All: Búsqueda Local a todos los individuos

Algorithm 4: AM-All: Aplicar BL a toda la población

Entrada: población P, problem, maxevals_local

```
1 mientras evals < maxEvals hacer
      // Generación (esquema generacional AGG-BLX)
      // ...
2     si generaciones mód 10 = 0 entonces
3         para i ← 1 to |P| y evals < maxEvals hacer
4             budget ← mín(maxevals_local, maxEvals − evals);
5             result ← BL(problem, P[i], budget);
6             P[i] ← result;
7             evals ← evals + result.evals;
```

AM-All aplica BL a tantos individuos como el presupuesto permita (típicamente ≈ 90 de los 100 en la primera fase). Concentra toda la explotación local en una etapa con poca evolución genética previa.

AM-Rand: Búsqueda Local aleatoria (10 % de población)

Algorithm 5: AM-Rand: Aplicar BL a 10 % aleatorio

Entrada: población P, problem, maxevals_local

mientras $evals < maxEvals$ **hacer**
  `// Generación (esquema generacional AGG-BLX)`
  `// ...`
  si *generaciones* mód $10 = 0$ **entonces**
    para $i \leftarrow 1$ **to** $|P|$ ***y*** $evals < maxEvals$ **hacer**
      si $rand() \leq 0{,}1$ **entonces**
        $budget \leftarrow$ mín($maxevals_local$, maxEvals $- evals$);
        result $\leftarrow$ BL(problem, $P[i]$, $budget$);
        $P[i] \leftarrow$ result;
        $evals \leftarrow evals +$ result.evals;

AM-Rand selecciona aleatoriamente aproximadamente el 10 % de la población (esperanza matemática: ≈ 10 individuos de 100) para aplicar BL, balanceando exploración genética con explotación local.

AM-Best: Búsqueda Local a los $0{,}1 \cdot N_p$ mejores

Algorithm 6: AM-Best: Aplicar BL a élite

Entrada: población P, problem, maxevals_local

mientras $evals < maxEvals$ **hacer**
  `// Generación (esquema generacional AGG-BLX)`
  `// ...`
  si *generaciones* mód $10 = 0$ **entonces**
    sort(P, descendente por fitness);
    para $i \leftarrow 1$ **to** mín($0{,}1 \cdot |P|$, $|P|$) ***y*** $evals < maxEvals$ **hacer**
      $budget \leftarrow$ mín($maxevals_local$, maxEvals $- evals$);
      result $\leftarrow$ BL(problem, $P[i]$, $budget$);
      $P[i] \leftarrow$ result;
      $evals \leftarrow evals +$ result.evals;

AM-Best aplica BL solamente a los $0{,}1 \cdot N_p = 10$ mejores individuos, focalizando el presupuesto de explotación en las soluciones más prometedoras.

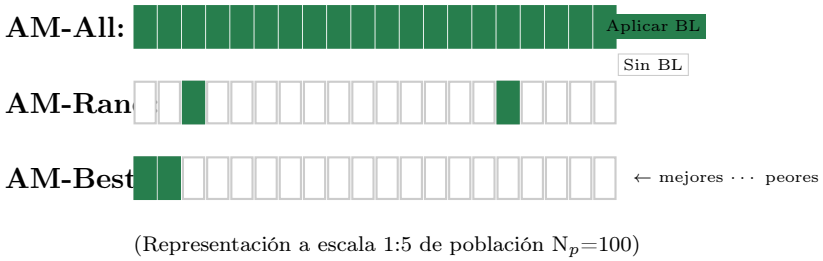


Figura 3: Comparación de estrategias de selección en AMs. Cada célula representa 5 individuos.

4. Estructura del Código y Manual de Usuario

4.1. Estructura de directorios

```
portfolio/
  main.cpp                  # Experimentos de 50 semillas
  CMakeLists.txt            # Sistema de construcción
  data/
    ibex_35.csv             # 30 empresas, 2015-2025
    syp_100.csv             # 97 empresas, 2015-2025
    syp_500.csv             # 457 empresas, 2015-2025
  common/
    mh.h, problem.h, random.hpp, util.h
  inc/
    portfolioProblem.h, randomSearch.h, greedy.h
    localsearch.h, genetic.h
  src/
    portfolioProblem.cpp, randomSearch.cpp, greedy.cpp
    localsearch.cpp, genetic.cpp
```

4.2. Compilación y ejecución

```
cmake .
make
./main [max_evals] [n_seeds]
```

Ejemplos: `./main` (10000 evals, 50 semillas), `./main 5000 30` (5000 evals, 30 semillas).

5. Experimentos y Resultados

5.1. Casos del problema

Fichero	**Empresas** (N)	**Días train**	**Días test**	**Rango** $[lo, hi]$
`ibex_35.csv`	30	2559	254	[0,005, 0,08]
`syp_100.csv`	97	2516	249	[0,005, 0,05]
`syp_500.csv`	457	2516	249	[0,005, 0,02]

Parámetros comunes: $\lambda = 500$, MAX_EVALS = 10 000, 50 semillas (1–50).

5.2. Resultados

Cuadro 1: Resultados IBEX 35 — $N = 30$, $[lo, hi] = [0{,}005, 0{,}08]$ (medias, 50 semillas)

Algoritmo	**Fit.Train**	**Fit.2025**	**Ben.2025**	**Evals**	**Tiempo (s)**
Greedy	−4,6495	−3,3868	0,1152	1,0	0,000
Random	−4,9411	−3,3060	0,0360	10000,0	0,024
BL	−4,4912	−3,0850	0,0293	2115,8	0,001
AGG-Arit	−4,7065	−3,1564	0,0408	10000,6	0,006
AGG-BLX	−4,5586	−3,1134	0,0377	10000,6	0,008
AGE-Arit	−4,6010	−3,1188	0,0347	10000,7	0,008
AGE-BLX	−4,5205	−3,1024	0,0342	10000,7	0,008
AM-All	−4,8652	−3,2068	0,0473	10000,0	0,025
AM-Rand	−4,5161	−3,1009	0,0381	10000,2	0,016
AM-Best	−4,5026	−3,0980	0,0373	10000,4	0,016

Cuadro 2: Resultados S&P 100 — $N = 97$, $[lo, hi] = [0{,}005, 0{,}05]$ (medias, 50 semillas)

Algoritmo	**Fit.Train**	**Fit.2025**	**Ben.2025**	**Evals**	**Tiempo (s)**
Greedy	−3,6635	−5,0204	0,0099	1,0	0,000
Random	−3,3904	−4,1131	0,0799	10000,0	0,064
BL	−2,7911	−3,7565	0,1088	10000,0	0,039
AGG-Arit	−3,0938	−3,8300	0,0868	10000,5	0,030
AGG-BLX	−3,0739	−3,8198	0,0851	10000,5	0,037
AGE-Arit	−3,0304	−3,8065	0,0888	10000,7	0,029
AGE-BLX	−2,9684	−3,7770	0,0903	10000,5	0,034
AM-All	−3,4100	−4,0801	0,0743	10000,0	0,221
AM-Rand	−3,1122	−3,8244	0,0809	10000,2	0,140
AM-Best	−3,0789	−3,8091	0,0857	10000,6	0,133

Cuadro 3: Resultados S&P 500 — $N = 457$, $[lo, hi] = [0{,}005, 0{,}02]$ (medias, 50 semillas)

Algoritmo	**Fit.Train**	**Fit.2025**	**Ben.2025**	**Evals**	**Tiempo (s)**
Greedy	−3,4384	−5,2046	−0,0628	1,0	0,000
Random	−3,6419	−4,4297	0,0123	10000,0	1,734
BL	−2,6282	−3,9162	0,0232	10000,0	1,168
AGG-Arit	−2,9723	−3,9803	0,0170	10000,5	0,656
AGG-BLX	−2,8710	−3,9238	0,0192	10000,8	0,673
AGE-Arit	−2,9693	−3,9774	0,0163	10000,6	0,654
AGE-BLX	−2,8756	−3,9346	0,0184	10000,7	0,663
AM-All	−3,4839	−4,2879	0,0155	10000,0	2,111
AM-Rand	−2,9504	−3,9267	0,0223	10000,2	1,542
AM-Best	−2,9334	−3,9337	0,0183	10000,6	1,326

5.3. Boxplots

Las Figuras 4–6 muestran la distribución del fitness de entrenamiento en las 50 ejecuciones para cada conjunto de datos.

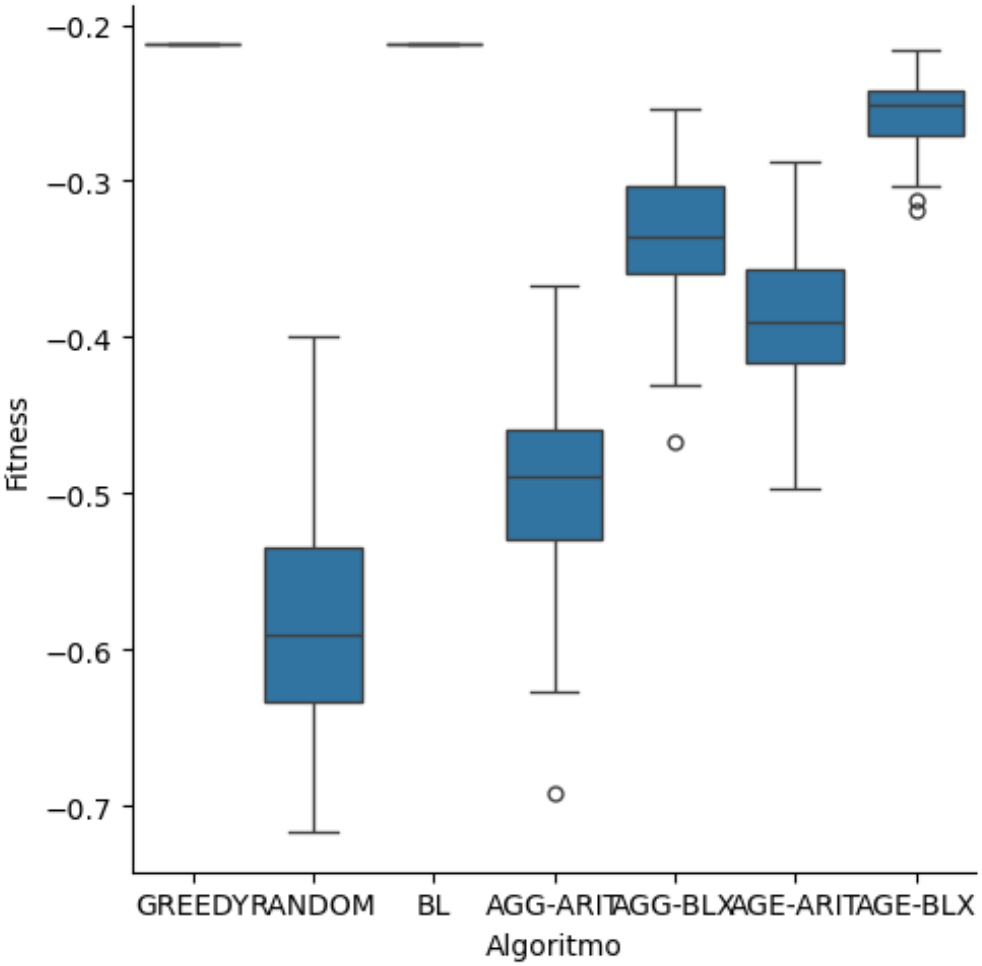


Figura 4: Distribución del fitness (2015–2024) en IBEX 35.

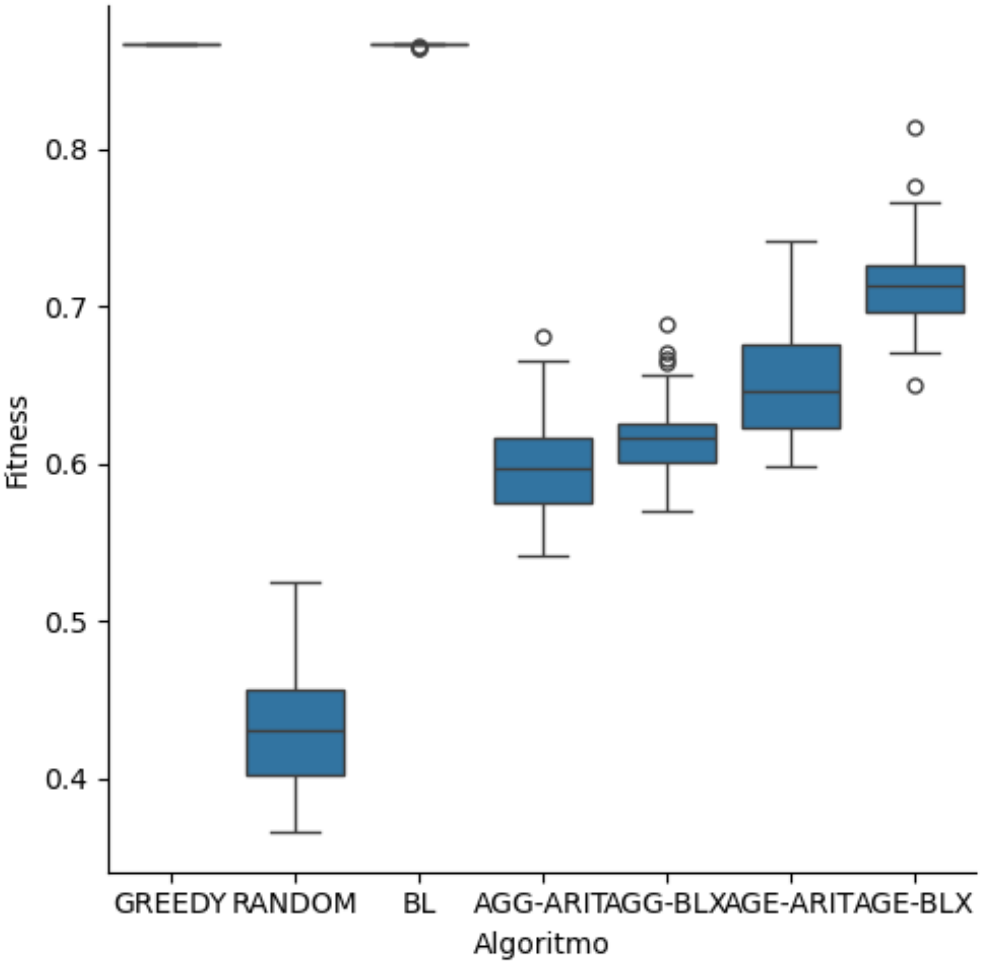


Figura 5: Distribución del fitness (2015–2024) en S&P 100.

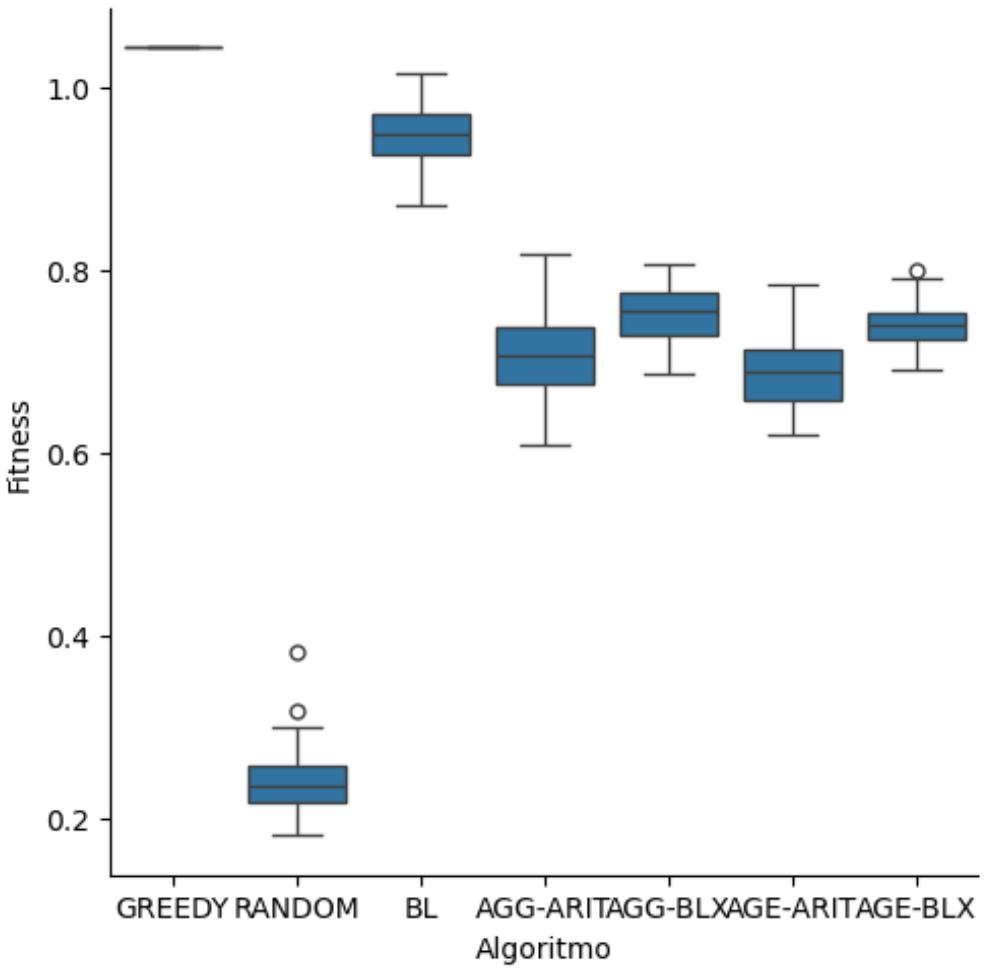


Figura 6: Distribución del fitness (2015–2024) en S&P 500.

6. Análisis de Resultados

6.1. Impacto del operador de cruce: AGG-Arit vs AGG-BLX

El cruce BLX-α con $\alpha = 0{,}3$ genera descendientes que pueden extenderse **más allá** del intervalo entre los padres, permitiendo explorar nuevas regiones del espacio. El cruce aritmético, al ser una combinación lineal, permanece siempre dentro del intervalo convexo de los padres.

Dataset	**AGG-Arit**	**AGG-BLX**	**Mejora BLX**
ibex_35 (N=30)	−4,7065	−4,5586	0,148
syp_100 (N=97)	−3,0938	−3,0739	0,020
syp_500 (N=457)	−2,9723	−2,8710	0,101

Observación clave: BLX supera a Aritmético en todos los datasets. La ventaja es más pronunciada en problemas pequeños (ibex_35) y grandes (syp_500). En espacios pequeños, BLX escapa más fácilmente de óptimos locales; en espacios grandes, la libertad exploratoria de BLX es crítica para encontrar soluciones competitivas.

El cruce aritmético es más explotativo (refina dentro de la región muestreada), mientras

que BLX es más explorativo (descubre nuevas regiones del espacio). Estos resultados justifican que los AMs usen **AGG-BLX** como base.

6.2. Impacto del operador de cruce: AGE-Arit vs AGE-BLX

El mismo patrón se observa en el esquema estacionario:

Dataset	AGE-Arit	AGE-BLX	Mejora BLX
ibex_35	−4,6010	−4,5205	0,081
syp_100	−3,0304	−2,9684	0,062
syp_500	−2,9693	−2,8756	0,094

BLX mejora consistentemente AGE en todos los datasets. Esto confirma que el operador de cruce es un factor crítico independiente del esquema de reemplazamiento.

6.3. Comparación de esquemas: AGG vs AGE

Usando BLX para una comparación justa:

Dataset	AGG-BLX	AGE-BLX	Ganador
ibex_35	−4,5586	−4,5205	AGE (+0,038)
syp_100	−3,0739	−2,9684	AGE (+0,106)
syp_500	−2,8710	−2,8756	AGG (+0,005)

Conclusión: AGE es superior a AGG en problemas de tamaño pequeño a medio. En problemas grandes (syp_500), son prácticamente equivalentes (diferencia de solo 0,005).

Razón algorítmica: El esquema generacional AGG reemplaza **toda** la población cada generación, causando *drift genético*: pérdida de soluciones buenas que no fueron seleccionadas en esa generación, pese al elitismo. AGE, por el contrario, mantiene 98 individuos intactos y solo reemplaza los 2 peores, conservando naturalmente una élite implícita. Con un presupuesto limitado (10000 evals), la estrategia conservadora de AGE es superior.

En problemas grandes, el drift en AGG queda compensado por la exploración masiva que permite (50 parejas nuevas por generación vs. 2 en AGE).

6.4. Algoritmos Meméticos: AM-All vs AM-Rand vs AM-Best

	ibex_35		syp_100		syp_500	
AM	Fit.Train	Evals	Fit.Train	Evals	Fit.Train	Evals
AM-All	−4,8652	10000,0	−3,4100	10000,0	−3,4839	10000,0
AM-Rand	−4,5161	10000,2	−3,1122	10000,2	−2,9504	10000,2
AM-Best	−4,5026	10000,4	−3,0789	10000,6	−2,9334	10000,6

Hallazgo sorprendente: **AM-All es peor que el mejor AG en todos los datasets**. Ibex 35: AM-All (−4,8652) vs AGE-BLX (−4,5205), diferencia: −0,345.

Causa: AM-All aplica BL a ≈ 90 individuos en la primera fase memetica (generación 10), agotando prácticamente todo el presupuesto de 10000 evaluaciones. El componente genético apenas evoluciona (solo 10 generaciones con ≈ 1000 evals). Los individuos todavía

inmaduros quedan atrapados en óptimos locales por la BL prematura. El resultado es peor que evolucionar solo genéticamente.

AM-Best es superior: Con solo 10 individuos por trigger, la BL es más selectiva. Los mejores individuos se refinan a lo largo de múltiples fases meméticas intercaladas con evolución genética. El componente genético tiene tiempo para diversificar.

AM-Rand vs AM-Best: AM-Rand selecciona aleatoriamente ≈ 10 individuos. AM-Best selecciona los mejores 10. AM-Best obtiene resultados ligeramente superiores en todos los datasets (ibex_35: −4,5026 vs −4,5161, diferencia +0,013), confirmando que focalizar la explotación local en las mejores soluciones es más eficiente.

6.5. Búsqueda Local vs Algoritmos Genéticos

La BL domina en todas las dimensiones:

Dataset	BL	Mejor AG	Mejor AM	Δ(BL−AG)
ibex_35	−4,4912	AGE-BLX (−4,5205)	AM-Best (−4,5026)	+0,029
syp_100	−2,7911	AGE-BLX (−2,9684)	AM-Best (−3,0789)	+0,177
syp_500	−2,6282	AGG-BLX (−2,8710)	AM-Best (−2,9334)	+0,243

El gap crece dramáticamente con el tamaño del problema. En syp_500, BL es 0,24 puntos mejor que el mejor AG, diferencia muy significativa.

Razones:

1. **Operador problem-tailored**: La transferencia del 15 % de peso entre activos respeta las restricciones del portfolio y navega eficientemente el espacio factible. Los operadores genéricos (cruce/mutación) son menos especializados.
2. **Presupuesto evaluativo**: Con 10000 evals y 100 individuos, el AG tiene ≈ 50 generaciones de evolución. La BL usa todos los 10000 evals en una única trayectoria de mejora monotónica.
3. **Convergencia de la BL**: En ibex_35, la BL converge al óptimo local en tan solo ≈ 2100 evals. En syp_100 y syp_500, agota las 10000 evals sin converger, lo que indica que el espacio es rico en mejoras locales accesibles.

6.6. Capacidad de generalización

Fitness de test (año 2025):

Dataset	BL (test)	Mejor AG (test)
ibex_35	−3,0850	AGE-BLX (−3,1024)
syp_100	−3,7565	AGE-BLX (−3,7770)
syp_500	−3,9162	AGG-BLX (−3,9238)

Observación: La BL no solo obtiene mejor fit.train sino también mejor fit.test, lo que indica que produce soluciones más **robustas** con mejor **generalización**.

Posible explicación: la trayectoria de mejora de la BL, al aplicar movimientos pequeños y suaves (15 % de peso), tiende a converger hacia soluciones que respetan las correlaciones entre activos, mientras que los cruzamientos y mutaciones genéticos pueden introducir discontinuidades que sobreajustan al periodo de entrenamiento.

6.7. Variabilidad y coste computacional

Algoritmo	ibex_35	syp_100	syp_500
Random	0,024s	0,064s	1,734s
BL	0,001s	0,039s	1,168s
AGG-BLX	0,008s	0,037s	0,673s
AGE-BLX	0,008s	0,034s	0,663s
AM-Best	0,016s	0,133s	1,326s

Análisis de tiempos:

- En ibex_35 (N=30), todos los algoritmos son rápidos ($\leq$ 0,025s). La evaluación del fitness es O(N^2) $\approx$ 900 operaciones.
- En syp_100 (N=97), la fitness es O(9409). Los AGs son ligeramente más rápidos que BL (0.034–0.037s vs 0.039s), ya que el operador de movimiento de la BL incurre en más overhead por evaluación que el cruce genético.
- En syp_500 (N=457), la fitness es O(208849). AM-All es el más lento (2.111s) por la BL intensiva aplicada a $\approx$ 90 individuos. Random (1.734s) es lento al generar 10000 soluciones completas desde cero. BL (1.168s) aplica movimientos incrementales baratos pero evalúa 10000 veces. Los AGs puros (0.654–0.673s) son los más rápidos al no incurrir en el overhead de la BL.

Variabilidad: Los boxplots (Figuras 4–6) muestran que:

- **Random**: Alta varianza (muy disperso).
- **Greedy**: Sin varianza (determinístico).
- **BL**: Varianza moderada a baja.
- **AGs/AMs**: Varianza baja (el promediado de población reduce el ruido).

Los AGs tienden a ser más consistentes (menor varianza) que BL gracias al efecto de averaging de la población, pero con peor valor medio.

6.8. Conclusiones generales

1. **BLX supera a Aritmético** en ambos esquemas. La exploración fuera del convex hull es crítica para todos los tamaños de problema.
2. **AGE supera a AGG** en presupuestos limitados. La conservación de élite implícita es más eficiente que el reemplazo generacional.
3. **AM-Best y AM-Rand son competitivos pero no dominantes**. AM-All fracasa al concentrar todo el presupuesto de BL en la primera fase memetica con población inmadura.
4. **BL domina globalmente**. El operador problem-tailored y la convergencia monotónica son decisivos con presupuestos limitados.
5. **Escalabilidad**: El gap BL–AG crece con N. En problemas grandes, los métodos especializados ganan con claridad.